

Technical Specification and Code Guide

How the final project requirements were implemented

Prepared 6 October 2026 · Systems Integration and Architecture · Mobile and Web Application Track

This guide connects the original project specification to the actual CareSync implementation. It explains the technical decisions, identifies the code that implements them, and separates implemented features from evidence that still needs to be collected. Use it alongside the final paper during the technical defense.

Reviewed web/backend revision: `b48083ee385ecb173bf2bbb853c7469b858324a6`. Code links open that exact GitHub revision and line. Short link labels show the filename and line; source excerpts also give the full path. Line numbers beside excerpts are annotations, not part of the executable code.

Mobile scope: the separate local `C:\Users\Adrian\Documents\CareSync_mobile` working copy, including existing uncommitted changes. Mobile references identify local files and lines; they do not claim that these changes are already committed. File hashes and an excerpt register are included in `technical-specification-source-index.json`.

Assessment source: the 14-page `Final Project Specification_MWA.pdf`, SHA-256

`b45a62c585bbe8418e66cec0c6f0c2329165aec8eb137880de08cb46f7b596`. Its requirements are assessment criteria, not authorization to operate the database or cloud services.

Evidence boundary: this document was prepared through source inspection and existing dated records. No database backup, restore, reset, email send, deployment or live clinical transaction was performed to write it. The older final paper and coverage report contain earlier implementation baselines; this guide updates the technical explanation for GridFS, Gmail API and the deployed topology. It does not finalize the group's title page, contributions or remaining evidence.

Contents

1. [What each specification requires and where CareSync fits](#)
2. [The ten required components, mapped to real code](#)
3. [Architecture and the shared integration contract](#)
4. [Authentication, roles and object-level access](#)
5. [Email verification and password recovery](#)
6. [Appointment workflow, concurrency and compensation](#)
7. [MongoDB structure, schedules and queue integration](#)
8. [Uploads, protected downloads and actual version history](#)
9. [Events, Gmail, comments, inbox and audit](#)
10. [Reports, validation and failure handling](#)
11. [Deployment, the mobile build and backup requirements](#)
12. [Testing: what exists, what passed and what still needs evidence](#)
13. [Functional, quality and screen traceability](#)
14. [Required paper chapters, diagrams and submission evidence](#)
15. [How to use this guide in the technical defense](#)

1. What each specification requires and where CareSync fits

“Implemented” means a code path exists. “Recorded verification” means an earlier dated check is documented. “Pending” means a requirement or its submission proof is not yet complete.

Specification heading / source page	CareSync response and evidence boundary
1. Project Overview / p.1	Integrated appointment, queue and consultation workflow, rather than only CRUD. Web and Flutter consume one API; MongoDB persists shared records; Gmail supplies an external service. See sections 3–10.
2. General Objective / p.1	Prototype, architecture, APIs, data, security and deployment are represented in source. Final presentation and full live evidence remain group tasks.
3. Specific Objectives / p.1	Current/target process, stakeholders, architectural choices, integration, validation, logs and tests are covered in the paper/code. Clinic confirmation, backup/restore and full E2E evidence remain pending.
4. Approved Themes / p.2	Theme 4: Healthcare Appointment, Queue, and Patient Notification Integration System. Patient app, clinic web, doctor schedules, queue, email and consultation records correspond directly. SMS is not implemented.
5. Minimum Components / pp.2–3	All ten have healthcare equivalents, mapped in section 2. Appointment is the work item; medical document is the submitted asset.
6. Integration Requirement / p.3	REST API connects two clients to clinic services. Gmail is an external API; booking/slot events automate notifications, audit and queue assignment. Only one option is required; CareSync does not claim ETL, ERP or a webhook.
7. Architecture Styles / pp.3–4	Layered modular backend with process-local event handlers. It is a single backend application, not deployed microservices. See section 3.
8. Integration Patterns / p.4	Shared backend/API hub for both clients, direct Gmail connection and internal event-driven reactions. Comparison and tradeoffs appear in section 3.
9. Required Documentation / pp.4–9	Existing 15-chapter paper is the main draft; section 14 maps its subsections to code/evidence. Update historical claims and complete placeholders.
10. Functional Requirements / pp.9–10	Section 13 provides 12 assessable implementation requirements; the existing paper has 17. Keep one consistent final FR register.
11. Non-functional Requirements / p.10	Section 13 provides 10 quality requirements and their actual verification boundaries. Backup and measured performance are not established by code alone.
12. Sample User Roles / p.10	Patient, Doctor, Staff and Admin adapt the generic role examples to the approved healthcare theme. Server permissions appear in section 4.
13. Required Diagrams / p.11	Eight required diagram assets plus an ERD exist. Section 14 maps them and identifies revisions needed for cloud storage/deployment.
14. Required Screens / p.11	All twelve have page/component/dialog counterparts in section 13. Instructor acceptance of consolidated screens must be confirmed.
15. Integration Evidence / p.11	Capture related request, persisted record, status/event, notification and audit result for the same demo transaction. Existing transport/HTTP checks alone are not the complete proof.
16. Required Testing Evidence / p.12	Minimum 8 functional, 5 integration, 5 error, 3 access-control and 1 E2E. Section 12 maps proposed case IDs to actual check files; final case-level results and live E2E proof need completion.
17. Suggested Group Roles / p.12	Assign actual responsibilities and contributions; do not infer people or completed work from Git history alone.
18. Suggested Timeline / p.12	Record actual milestones, owners and completion dates. No invented dates in this guide.
19. Final Submission / p.13	Paper/PDF, prototype, source, DB/migration material, feature screenshots, integration proof, tests, backup/recovery, slides and contributions. Several evidence items remain pending.
20. Presentation and Defense / p.13	Explain the paths and decisions in this guide; rehearse booking through doctor completion across web/mobile. See section 15.
21. Important Reminders / pp.13–14	Use demo data, protect credentials and patient data, keep architecture and diagrams accurate, and understand the code being defended.

The specification contains generic creative-production examples as well as an explicitly approved healthcare theme. This guide maps their component purposes to healthcare; it does not claim CareSync contains animation or creative campaign features. Confirm this theme-based interpretation with the instructor.

2. The ten required components, mapped to real code

Required component	Healthcare implementation	Main code
User Management	Patient registration/login, clinic accounts, role/status management and password verification.	user.controller.js:18 ; user.service.js:133 ; AddUserModal.jsx:1
Project / Production	Appointment is the work item: create, assign, review, check in, consult and complete.	appointment.service.js:89 ; StaffDashboard.jsx:1
Asset / File Submission	Authorized clinic user attaches categorized medical documents to an appointment.	upload.js:115 ; ConsultationModal.jsx:1
Version Tracking	Status/booking history plus note revisions and retained document snapshots.	Appointment.js:59 ; appointment.service.js:675 ; VersionHistory.jsx:1
Review / Approval	Staff/Admin approve or request booking correction; authorized clinic users decline; patient resubmits.	appointment.routes.js:33 ; appointment.service.js:260
Comment / Feedback	Appointment-scoped comments with author, role and timestamp.	appointment.service.js:813 ; AppointmentComment.js:1 ; AppointmentComments.jsx:1
Notification / Log	Private inbox, read state, selected patient emails and delivery outcome.	notification.service.js:8 ; NotificationInbox.jsx:1
Dashboard / Report	Four role dashboards; date/status/doctor aggregates for clinic users.	App.jsx:83 ; report.service.js:5
Audit Log	Events recorded with actor, target and changes; Admin-only viewing.	auditLog.handler.js:1 ; auditLog.routes.js:17 ; AuditLogViewer.jsx:1
Integration Component	Shared REST API, Gmail HTTPS API, internal workflow events and queue automation.	index.js:52 ; email.service.js:43 ; slotFreed.handler.js:1

Scope distinction for the defense: review/approval concerns the appointment booking and requested corrections. Documents have upload/replacement/version history, but there is no separate per-document approved/rejected publication workflow. If the instructor expects that specific interpretation of asset approval, obtain acceptance of the healthcare mapping rather than describe a nonexistent file approval feature.

3. Architecture and the shared integration contract

Why the application is layered

React and Flutter handle presentation. Express routes select operations; middleware checks identity and permissions; controllers translate HTTP into service calls; services enforce clinic rules; Mongoose models persist data. The event layer reacts to completed service actions. Routes/controllers/services are directories within one deployed Node process.

Layer	Example / responsibility
Web presentation	BookAppointmentModal.jsx:1 renders booking choices.
Mobile presentation	CareSync_mobile/lib/booking_screen.dart:1 (local working copy) implements the patient booking flow.
Client transport	api.js:21 and CareSync_mobile/lib/api_services.dart:1 (local working copy) encode requests and bearer tokens.

Layer	Example / responsibility
Route and access control	appointment.routes.js:18 restricts booking to Patient.
Controller	appointment.controller.js:31 derives patient identity from the authenticated request.
Business service	appointment.service.js:89 resolves slots, reserves capacity, writes history and emits the event.
Persistence	Appointment.js:21 and Slot.js:69 define stored structure and indexes.
Event reactions	server.js:66 registers notification, audit and freed-slot listeners after startup checks.

One request across web, mobile and database

Illustrative payload, not a live execution result:

```
{ "slotId": "aaaaaaaaaaaaaaaaaaaaaa", "reason": "Demo consultation" }
```

The client sends POST /api/appointments with Authorization: Bearer <session token>. The server verifies the session, requires Patient, and assigns patientId from req.user, not from a client-supplied patient ID. For a selected slot it derives doctor/date/time from the slot record. The service writes a Pending appointment with history. The controller responds with HTTP 201 and {success:true,data:...}. Clinic users subsequently read the same appointment from the same API/database.

Source: [appointment.controller.js:31](#). Full path: backend/src/controllers/appointment.controller.js.

```
31 async function create(req, res, next) {
32   try {
33     const { doctorId, date, timeSlot, slotId, reason } = req.body;
34     const appointment = await appointmentService.create({
35       patientId: req.user.id,
36       doctorId,
37       date,
38       timeSlot,
39       slotId,
40       reason,
41     });
42     res.status(201).json({ success: true, data: appointment });
43   } catch (err) {
44     next(err);
45   }
}
```

Source: [api.js:21](#). Full path: frontend/src/services/api.js.

```
21 async function request(endpoint, options = {}) {
22   const token = getStoredToken();
23   const isFormData = options.body instanceof FormData;
24
25   const headers = {
26     ...(isFormData ? {} : { 'Content-Type': 'application/json' }),
27     ...(token ? { Authorization: `Bearer ${token}` } : {}),
28     ...options.headers,
29   };
30
31   const config = {
32     ...options,
```

VITE_API_BASE_URL chooses the web API base at build time. Flutter's API_BASE_URL is also a build-time define; localhost on a phone points to the phone, so the deployed APK uses the hosted Render endpoint.

Source: **CareSync_mobile/lib/api_services.dart:17** (local working copy). Full path: CareSync_mobile/lib/api_services.dart.

```

17 static const baseUrl = String.fromEnvironment(
18   'API_BASE_URL',
19   defaultValue: 'http://127.0.0.1:5000/api',
20 );
21 static const _storage = FlutterSecureStorage();
22 static final sessionExpired = ValueNotifier(false);
23

```

Architecture and pattern comparison

Option	Decision and tradeoff
Layered architecture	Selected: clear HTTP/business/persistence responsibilities, simple single-service deployment. Changes to service rules serve both clients.
Service-oriented architecture	External Gmail and REST boundaries demonstrate service integration; internal modules are not independent deployed services.
Microservices	Not selected: would add coordination, hosting and operational complexity for this prototype.
Event-driven architecture	Used internally for notifications, audit and queue reassignment. Events are process-local; no durable broker/outbox or replay.
Point-to-point integration	Backend directly calls Gmail. Straightforward for one external provider, but provider-specific code remains in email service.
Hub-and-spoke integration	Both clients use the shared backend as their integration hub. They do not connect to Atlas directly. Backend outage affects both.
Shared database pattern	Backend modules share MongoDB. Clients share business data through the API, not unrestricted direct database credentials.

The startup sequence waits for MongoDB/model indexes, refuses unresolved assignment recovery and materializes schedule plans before listening: [server.js:46](#). This supports predictable initialization, but is not proof of uptime or load capacity.

4. Authentication, roles and object-level access

Authentication is enforced by the API

The central router has an exact public allowlist for login, registration/password verification and the public clock/queue. Everything else runs protect. UI visibility does not grant access. Public self-registration explicitly sets `role: 'Patient'`; only the Admin route can create other account types.

Source: [index.js:32](#). Full path: `backend/src/routes/index.js`.

```

32 const publicRequests = new Set([
33   'POST /users/login', 'POST /users/register',
34   'POST /users/register/otp', 'POST /users/password/otp', 'POST /users/password/reset',
35   'GET /system/time', 'HEAD /system/time',
36   'GET /walkins/now-serving', 'HEAD /walkins/now-serving',
37 ]);
38 router.use((req, res, next) => {
39   res.set('Cache-Control', 'no-store');
40   res.set('X-Content-Type-Options', 'nosniff');
41   const route = `${req.method} ${req.path.replace(/\/+$/, '').toLowerCase()}`;
42   if (publicRequests.has(route)) return next();
43   return protect(req, res, next);
44 });
45 router.use((req, res, next) => {

```

Passwords are hashed with bcrypt at cost 10 on save, compared through comparePassword, and normally excluded from queries: [User.js:87](#). Login signs a JWT containing account ID and token version, with configured expiry: [user.service.js:174](#). protect verifies HS256, fetches the current account and rejects deactivated/unrecognized accounts or stale token versions.

Source: [auth.js:60](#). Full path: backend/src/middleware/auth.js.

```
60     const decoded = jwt.verify(token, env.JWT_SECRET, { algorithms: ['HS256'] });
61     if (!mongoose.isObjectIdOrHexString(decoded.id)) throw new Error('Invalid token subject');
62
63     // Attach user to request
64     let user;
65     try { user = await User.findById(decoded.id); }
66     catch (err) { return next(err); } // A database outage does not invalidate a login token.
67     if (!user || user.status === 'Deactivated' || !ROLES.includes(user.role) ||
68         (decoded.version ?? 0) !== (user.tokenVersion || 0)) {
69         return res.status(401).json({
70             success: false,
71             message: 'Account is unavailable or deactivated',
```

Fetching the current account means role/status changes take effect on later requests. A database outage is forwarded as a service error rather than automatically invalidating the user's token. Password reset hashes the new password and increments tokenVersion, invalidating older sessions: [user.service.js:180](#).

Roles and ownership are separate checks

Source: [auth.js:19](#). Full path: backend/src/middleware/auth.js.

```
19 function assertAppointmentAccess(appointment, user) {
20     const allowed = user && (['Staff', 'Admin'].includes(user.role) ||
21         (user.role === 'Patient' && recordId(appointment.patient) === recordId(user._id || user.id)) ||
22         (user.role === 'Doctor' && recordId(appointment.doctor) === recordId(user._id || user.id)));
23     if (!allowed) throw Object.assign(new Error('You do not have permission to access this appointment'), {
24         statusCode: 403 });
25 }
```

This is the object-level control: Patient must own the appointment; Doctor must be assigned; Staff/Admin can access clinic appointments. It runs before individual appointment operations, including upload/download. Route-specific roles further restrict mutations.

Capability	Patient	Doctor	Staff	Admin
Self-book / resubmit	Own booking	No	No	No
Read appointment, comments, history and downloads	Own	Assigned	Clinic	Clinic
Approve / request correction / assign slot	No	No	Yes	Yes
Decline / cancel	Own cancellation, subject to state	Assigned, subject to state	Subject to state	Subject to state
Check in / start / complete	No	Assigned	Yes	Yes
Notes / upload / replace / archive document	No	Assigned	Yes	Yes
Reports	No	Own consultations	Clinic	Clinic
Account role/status administration	No	No	No	Yes
Audit viewer / clock override / demo reset	No	No	No	Yes
Notification inbox	Own	Own	Own	Own

Sources: [appointment.routes.js:23](#); [user.routes.js:25](#); [system.routes.js:31](#); [report.service.js:14](#). Exact status/date rules still apply; a role check alone is not permission to perform every transition.

Practical limitation: the current web token is in localStorage, not an HttpOnly cookie. Flutter uses secure storage. Source contains access controls, but there is no claim of a penetration test, regulatory certification or protection against all XSS/device compromise. Clinic Staff/Admin can edit clinical records; explain this broader prototype permission honestly when discussing least privilege.

5. Email verification and password recovery

Registration is two steps: request a code, then register with that code. The server normalizes the email, generates six digits using `crypto.randomInt`, and stores a keyed HMAC instead of plaintext code. Purpose and email are included in the hash to prevent a registration code being reused as a password-reset code.

Source: [otp.service.js:14](#). Full path: `backend/src/services/otp.service.js`.

```
14 function hashCode(purpose, email, code) {
15   // Keyed hashes prevent guessing six-digit codes from a database dump alone.
16   return createHmac('sha256', env.JWT_SECRET).update(JSON.stringify(['email-otp', purpose, email,
code])).digest('hex');
17 }
```

`reserveOtp` uses a conditional MongoDB update for a 60-second resend delay and fewer than five sends per hour. Codes expire after ten minutes. `consumeOtp` permits fewer than five incorrect attempts and atomically marks a correct challenge consumed. A second use cannot satisfy the same filter.

Source: [otp.service.js:80](#). Full path: `backend/src/services/otp.service.js`.

```
80 const filter = { _id: `${purpose}:${email}`, ready: true, consumedAt: null, expiresAt: { $gt: new Date() },
attempts: { $lt: 5 } };
81 const challenge = await EmailOtp.findOneAndUpdate({ ...filter, codeHash: hashCode(purpose, email, code) },
82   { $set: { consumedAt: new Date(), codeHash: null, ready: false } }, { new: true });
83 if (!challenge) {
84   await EmailOtp.updateOne(filter, { $inc: { attempts: 1 } });
85   throw fail('Invalid, expired, or used code. After five attempts, request a new code.');
```

Sources: [otp.service.js:19](#); [otp.service.js:42](#); [EmailOtp.js:17](#); [otpRateLimit.js:4](#).

The challenge becomes usable only after the email provider accepts the verification message. Failure returns 503 and invalidates its hash. Password-reset requests return a generic response for active/missing accounts; sending is asynchronous for active accounts. The expiry predicate is the validity check; the TTL index only cleans old records later.

The account endpoint limiter permits 60 requests per IP per 15 minutes and bounds its Map at 10,000 entries. It is process-local, so it resets on restart and is not shared across workers. Database challenge cooldown/attempt rules provide an additional independent control.

The shared web password field includes a Show/Hide control: [PasswordInput.jsx:1](#). This improves input usability; it does not change hashing or authorization. The current minimum password length is six characters, with a 72-byte upper bound for bcrypt; do not describe it as a stronger password policy than the code enforces.

Evidence still required: use a demo email to receive a real deployed signup/password code, complete the flow, and record the result without revealing the code or credentials. The deployment record confirms OAuth authentication, not actual inbox delivery.

6. Appointment workflow, concurrency and compensation

Lifecycle and review

The main lifecycle is Pending → Confirmed → Checked In → In Progress → Completed. Alternatives include Needs correction, Declined, Cancelled and No-show. The enum is [Appointment.js:21](#); allowed transitions are enforced in service methods, not by permitting arbitrary status strings from the client.

Action	Implementation / purpose
Book and reserve	appointment.service.js:89 ; selected doctor requires an actual slot. Flexible booking can remain unassigned until clinic assignment.
Approve	appointment.service.js:160 ; updates Pending conditionally and confirms the linked reservation.
Request correction / resubmit	appointment.service.js:260 ; authorized workflow, booking revision/history and stale-change checks.
Decline / cancel / no-show	appointment.service.js:209 ; appointment.service.js:331 ; appointment.service.js:381 . Reasons/history/events and linked slot changes.
Check in / clinical lifecycle	appointment.service.js:432 ; appointment.service.js:489 . Queue tickets and clinic-side status transitions.
Automatic no-show processing	appointment.service.js:30 . Invoked by relevant request/service flows and simulated clock changes; not a separately deployed scheduler.

Two users cannot successfully claim the same available slot

The pre-read supplies helpful validation; the decisive operation is the atomic conditional update below. A competing request sees no matching Available/unowned slot and receives 409.

Source: [slot.service.js:304](#). Full path: backend/src/services/slot.service.js.

```
304   const reserved = await Slot.findOneAndUpdate(
305     { _id: slotId, status: 'Available', appointment: null },
306     { $set: { status: 'Reserved-Tentative', appointment: appointmentId, reservationOperation: operationId } },
307     { new: true }
308   );
309   if (!reserved) {
310     const err = new Error('This slot has already been taken');
311     err.statusCode = 409;
312     throw err;
313   }
314   return reserved;
315 }
```

The unique (doctor,date,startTime) index prevents duplicate slot records: [Slot.js:69](#). Booking creates an assignment journal before claiming the slot, then stores the appointment and marks the journal done. A failed write triggers compensation.

Source: [appointment.service.js:117](#). Full path: backend/src/services/appointment.service.js.

```

117   const appointmentId = new mongoose.Types.ObjectId();
118   const operation = slotId ? await recovery.begin({ kind: 'booking', appointment: appointmentId, slot: slotId
}) : null;
119   let appointment;
120   try {
121     if (slotId) await slotService.reserveSlot(slotId, appointmentId, operation._id);
122     appointment = await Appointment.create({
123       _id: appointmentId,
124       patient: patientId,
125       doctor: resolvedDoctorId,
126       slot: slotId || null,
127       date: resolvedDate,
128       timeSlot: resolvedTimeSlot,
129       reason,

```

What recovery does, and what it does not do

[assignmentRecovery.service.js:13](#) checks appointment/slot/walk-in references. If the assignment committed but acknowledgement failed, it marks the journal done. Otherwise it releases only records still owned by that operation. Ambiguous existing clinical records require reconciliation; it does not delete them to force success.

Source: [assignmentRecovery.service.js:31](#). Full path: backend/src/services/assignmentRecovery.service.js.

```

31   const results = await Promise.allSettled([
32     Slot.updateOne({ _id: operation.slot, appointment: operation.appointment, reservationOperation:
operation._id,
33     status: { $in: ['Reserved-Tentative', 'Reserved-Confirmed'] } },
34     { $set: { status: 'Available', appointment: null, reservationOperation: null } }),
35     ...(operation.walkIn ? [WalkIn.updateOne({ _id: operation.walkIn, appointment: operation.appointment,
36     assignedSlot: operation.slot, status: 'Slot Assigned' },
37     { $set: { status: 'Waiting', appointment: null, assignedSlot: null } })] : []),
38   ]);

```

These are guarded per-document writes plus compensating recovery, not an ACID transaction across all collections. Startup refuses pending recovery records; [recover-assignments.js:1](#) provides an offline maintenance path. This is **operation recovery**, not a database backup or disaster-recovery restore.

Corrections and note/file replacement use expected revision or current-value predicates. A stale client receives 409 instead of silently overwriting newer work. Tests for this logic are [concurrency.check.js:1](#) and [corrections.check.js:1](#). Successful isolated race checks support these paths; they are not a measured throughput benchmark.

7. MongoDB structure, schedules and queue integration

Data model

Model / storage	What it preserves and how it connects
User	Identity, bcrypt password, role/status, token version, working hours. User.js:1
Appointment	References patient or walk-in, doctor and slot; embeds status/booking history, notes and attachment versions. Appointment.js:1
Slot and SlotPlan	Concrete doctor/date/start/end capacity and recurring generation plans. Slot.js:1 ; SlotPlan.js:1
WalkIn	Queue ticket, daily key, status, assignment references and arrival. WalkIn.js:1
AppointmentComment	Separate appointment-linked comment records with author snapshots/timestamps. AppointmentComment.js:1
Notification	Recipient-owned inbox item and email-delivery outcome/read state. Notification.js:1
AuditLog	Action, actor, target, timestamp and change metadata. AuditLog.js:1

Model / storage	What it preserves and how it connects
EmailOtp	Verification challenge hash, validity, attempts, cooldown and TTL cleanup. EmailOtp.js:1
AssignmentRecovery	Durable pending/done journal for incomplete booking/assignment operations. AssignmentRecovery.js:1
medicalFiles.files / medicalFiles.chunks	GridFS metadata and actual uploaded bytes in the application database. fileStorage.service.js:5

Mongoose schemas enforce types/enums; references express relationships but are not SQL foreign-key constraints. Services must maintain relationships. Keep embedded history growth in mind: unlimited retained versions can eventually exceed MongoDB's per-document limit. The code does not claim unlimited archive capacity.

Doctor availability and clinic date rules

[slot.service.js:54](#) creates slots based on working hours/duration; [slot.service.js:147](#) materializes recurring plans. Clinic timezone is configured as Asia/Manila. [timeHelper.js:1](#) handles date/time validation; date-only appointment keys are kept distinct from timestamp instants. Clock simulation is Admin-controlled and process-local: [systemTime.js:1](#).

Walk-ins and scheduled arrivals share the public display

Walk-ins receive daily queue numbers with a unique (queueDay, queueNumber) index and duplicate-key retry: [WalkIn.js:70](#); [walkIn.service.js:52](#). Assignment conditionally claims the waiting walk-in and available slot, creates an appointment, and uses the same recovery journal.

A SLOT_FREED handler tries to assign a released eligible slot to the oldest waiting walk-in: [slotFreed.handler.js:1](#); [walkIn.service.js:206](#). Competing 409 claims are retried. Eligible date, operating hours and slot time are still checked.

The public result combines in-progress/waiting walk-ins with checked-in/in-progress scheduled appointments. Scheduled tickets use A- prefixes. The response maps entries to ticket and status, rather than patient names or medical reasons: [walkIn.service.js:336](#).

Source: [ClinicDisplayScreen.jsx:7](#). Full path: frontend/src/pages/ClinicDisplayScreen.jsx.

```
7 const POLL_INTERVAL = 10_000;
```

The web display polls every ten seconds; its clock advances locally between fetches. Although a handler accepts an optional Socket.io argument, [server.js:68](#) supplies none and does not initialize a Socket.io server. Describe current updates as polling with internal events, not live WebSocket delivery.

8. Uploads, protected downloads and actual version history

Why documents are in GridFS

Production selects `UPLOAD_STORAGE=gridfs`. Upload bytes are streamed into the `medicalFiles` bucket in MongoDB; appointment records retain document metadata/storage references. This lets web/mobile use one protected record and file source without depending on a persistent Render local folder. Local categorized disk storage remains a development option.

Source: [upload.js:115](#). Full path: backend/src/middleware/upload.js.

```

115 const gridfsStorage = {
116   _handleFile(req, file, callback) {
117     let stream;
118     try {
119       stream = getBucket().openUploadStream(file.originalname, { metadata: { appointment: req.params.id } });
120       pipeline(file.stream, stream, error => {
121         if (error) {
122           discardUpload({ storageId: stream.id }).catch(() => {}).finally(() => callback(error));
123           return;
124         }
125         callback(null, { storageId: stream.id, storageUrl: `/uploads/gridfs/${stream.id}`,
126           filename: file.originalname, size: stream.gridFSFile.length });
127       });

```

Limits are 25 MiB, one file, five fields, and 4,096 bytes per field: [upload.js:137](#). Extension allowlist includes PDF/images/Office/text/CSV. It is extension-based validation, not antivirus scanning or deep content validation. Failed unattached uploads are discarded; cleanup checks preserve references if an attachment commit succeeded but acknowledgement failed: [appointment.controller.js:138](#).

A storage reference is not a public download link

/uploads is deliberately blocked at the server. Downloads use /api/appointments/:id/documents/:docId/download, or the protected version-specific endpoint. Identity and appointment ownership are checked before resolving GridFS bytes. The controller selects metadata from the authorized appointment, sets attachment/no-store/nosniff headers and streams the matching file.

Sources: [server.js:31](#); [appointment.routes.js:59](#); [appointment.controller.js:199](#); [fileStorage.service.js:11](#). Local fallback also verifies path containment and real paths. The deployment migration retains legacy references through GridFS metadata.legacyUrl; it does not make them public URLs.

Notes and files have content history, not only status labels

Saving changed consultation notes increments revision and appends a snapshot containing notes, author and time. The update tests the earlier content/revision to reject a stale writer.

Source: [appointment.service.js:567](#). Full path: backend/src/services/appointment.service.js.

```

567 const snapshot = { version: consultationNotes.trim(), savedAt: new Date(), savedBy: actor._id ||
actor.id,
568   authorName: actorName(actor), authorRole: actor.role };
569 // ponytail: embedded history gives one atomic write; move to a transaction-backed collection if records
approach MongoDB's 16 MB limit.
570 const updated = await Appointment.findOneAndUpdate(
571   { _id: appointmentId, consultationNotes: appointment.consultationNotes ?? null,
572     notesRevision: revision ? revision : { $in: [0, null] } },
573   { $set: { consultationNotes: snapshot.notes, notesRevision: version },
574     $push: { noteVersions: { $each: [...!(appointment.noteVersions || []).length ? previous : []], snapshot]
575   } } },
576   { new: true, runValidators: true }
577 );
578 if (!updated) throw Object.assign(new Error('Notes changed while saving. Close and reopen the record.'), {
statusCode: 409 });
579 emitRecordChange(updated, actor, { operation: 'Notes updated', notesVersion: version });
580 return updated;

```

File replacement retains the document ID, increments its version and stores the previous URL/metadata as a snapshot: [appointment.service.js:585](#). Removing an attachment archives its metadata and removes the current item in one update; bytes remain available through version history.

Source: [appointment.service.js:631](#). Full path: backend/src/services/appointment.service.js.

```

631 // Archive and remove the current attachment together; the files remain available in version history.
632 const updated = await Appointment.findOneAndUpdate(
633   { _id: appointmentId, documents: { $elemMatch: { _id: document._id, url: document.url } } },
634   { $pull: { documents: { _id: docId } }, $push: { archivedDocuments: archived } }, { new: true,
runValidators: true }
635 );
636 if (!updated) throw Object.assign(new Error('Document was already removed; refresh the appointment'), {
statusCode: 409 });
637 emitter.emit(EVENTS.DOCUMENT_DELETED, {
638   performedBy: actor._id || actor.id, targetModel: 'Appointment', targetId: updated._id, archived: true,
639   document: { _id: document._id, filename: document.filename, type: document.type },

```

[appointment.service.js:675](#) combines current and archived entries. `getDocumentVersion` validates the requested version and returns only an actual stored snapshot. UI is [VersionHistory.jsx:1](#); mobile reads versions/downloads through [CareSync_mobile/lib/api_services.dart:274](#) (local working copy) and its appointment detail screen.

Legacy demo boundary: one historical demo attachment was already missing before migration. The user confirmed these demo files are disposable. Do not fabricate restored bytes or claim every legacy download succeeds. A new complete demo attachment is needed for the version/download demonstration.

9. Events, Gmail, comments, inbox and audit

Event-driven integration within the backend

Source: [emitter.js:14](#). Full path: `backend/src/events/emitter.js`.

```

14 const emitter = new EventEmitter();
15
16 // Increase listener limit for production (we'll have multiple handlers per event)
17 emitter.setMaxListeners(20);
18
19 // Track background writes so a demo reset cannot race an unfinished handler.
20 emitter.pending = new Set();
21 emitter.onAsync = (event, handler) => emitter.on(event, (...args) => {
22   const job = Promise.resolve(handler(...args));
23   emitter.pending.add(job);
24   job.then(() => emitter.pending.delete(job), () => emitter.pending.delete(job));
25   return job;

```

Business services emit named events from [events.js:1](#). Registered handlers create notifications/audit entries and attempt queue reassignment. The pending set lets demo reset detect unfinished background jobs. It is not durable storage of events. A process crash between the clinical write and handler persistence can lose a reaction; no automatic replay or distributed delivery guarantee is implemented.

External Gmail API integration

[email.service.js:8](#) refreshes OAuth access tokens using backend-only credentials, caches them before expiry and coalesces simultaneous refresh requests. `sendEmail` encodes MIME headers/body and submits a base64url message over HTTPS.

Source: [email.service.js:54](#). Full path: `backend/src/services/email.service.js`.

```

54     response = await fetch('https://gmail.googleapis.com/gmail/v1/users/me/messages/send', {
55       method: 'POST', signal: AbortSignal.timeout(15000),
56       headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
57       body: JSON.stringify({ raw: Buffer.from(mime).toString('base64url') }),
58     });
59     data = await response.json();
60   } catch { throw emailFailure('GMAIL_SEND_FAILED'); }

```

Only environment variable names appear in source/docs; actual secrets belong in Render settings. The retained SMTP branch is a separate configurable transport. Production uses Gmail API. A successful OAuth refresh proves authentication; a Gmail message ID proves provider acceptance; neither alone proves inbox delivery/read status.

Notification persistence and failure behavior

Source: [notification.service.js:8](#). Full path: backend/src/services/notification.service.js.

```

8  async function sendNotification({ userId, appointment, type, title, message }) {
9    if (!userId) return;
10   const eligible = env.EMAIL_ENABLED && PATIENT_EMAIL_TYPES.has(type);
11   // Save the inbox entry first. SMTP failures must not remove it or fail the clinical action.
12   const notification = await Notification.create({
13     user: recordId(userId), appointment: appointment ? recordId(appointment) : null, type, title, message,
14     emailDelivery: { status: eligible ? 'pending' : env.EMAIL_ENABLED ? 'skipped' : 'disabled', errorCode:
env.EMAIL_ENABLED && !eligible ? 'IN_APP_ONLY' : null },
15   });
16   if (!eligible) return notification;

```

An inbox item is saved before selected email delivery is attempted. Booking confirmed, declined, cancellation and no-show are patient email types. Other eligible workflow notifications can be in-app only. Delivery records track pending/sent/failed/skipped/disabled, timestamps, message ID and bounded error codes. Email failure does not remove the inbox entry or fail the already-completed clinic action. Sending is single-attempt, without durable outbox/replay: [notification.service.js:39](#).

Inbox reads/mark-read operations filter by req.user._id, even for Admin: [notification.routes.js:13](#). Repeated mark-read preserves the first timestamp.

Comments and audit are different records

[appointment.service.js:813](#) persists appointment feedback with author identity/role/name and emits the related action. Appointment access is required for reading and writing. [AppointmentComments.jsx:1](#) displays the discussion.

Audit handlers store important business events with actor/target/change metadata: [auditLog.handler.js:1](#); [auditLog.service.js:22](#). The API offers Admin-only read access and no ordinary edit route. “Audit trail” does not mean cryptographic tamper-proof storage: database administrators and Clear Demo Data can remove records; handler failure is logged, not durably replayed. Governance must define ownership, retention and who can access backups containing these records.

10. Reports, validation and failure handling

Reports aggregate shared operational data

Report requests validate the date range and doctor ID. Doctor role is forced to its own ID; Staff/Admin can view clinic data. MongoDB \$facet computes status totals, daily counts and per-doctor counts from the same filtered appointment set.

Source: [report.service.js:14](#). Full path: backend/src/services/report.service.js.

```

14   const targetDoctor = actor.role === 'Doctor' ? String(actor._id || actor.id) : doctorId;
15   if (targetDoctor) {
16     if (!mongoose.isObjectIdOrHexString(targetDoctor)) throw Object.assign(new Error('Invalid doctor ID'), {
statusCode: 400 });
17     match.doctor = new mongoose.Types.ObjectId(targetDoctor);
18   }
19   const [result] = await Appointment.aggregate([
20     { $match: match },
21     { $facet: {
22       statuses: [{ $group: { _id: '$status', count: { $sum: 1 } } }],
23       daily: [
24         { $group: { _id: { $dateToString: { format: '%Y-%m-%d', date: '$date', timezone: 'UTC' } } },
25           total: { $sum: 1 }, completed: { $sum: { $cond: [{ $eq: ['$status', 'Completed'] }, 1, 0] } } } } ],

```

The UI is [ReportsPage.jsx:1](#). Role dashboard selection is [App.jsx:83](#). Report totals are operational appointment counts; the code does not implement billing analytics or diagnose patients.

Errors preserve a usable contract

Source: [errorHandler.js:10](#). Full path: backend/src/middleware/errorHandler.js.

```

10   const statusCode = err.code === 'LIMIT_FILE_SIZE' ? 413 : err.name === 'MulterError' ? 400 :
11     err.statusCode || err.status || ([ 'CastError', 'ValidationError' ].includes(err.name) ? 400 : 500);
12   const message = err.code === 'LIMIT_FILE_SIZE' ? 'The file exceeds the 25 MB upload limit.' :
13     err.type === 'entity.parse.failed' ? 'The request body must be valid JSON.' :
14     err.type === 'entity.too.large' ? 'The request body is too large.' :
15     process.env.NODE_ENV === 'production' && statusCode >= 500 && !err.statusCode ? 'The service is temporarily
unavailable. Try again shortly.' : err.message || 'Internal Server Error';
16
17   // Log full error in development

```

Response	Concrete trigger / implementation
400	Invalid JSON/date/ID/query/validation; central error middleware and service checks.
401	Missing/invalid/stale/deactivated session. auth.js:44
403	Wrong role/appointment owner or disallowed browser origin. auth.js:89 ; server.js:22
404	Missing authorized record/file/version or blocked raw upload URL.
409	Taken slot, stale revision, duplicate account or unsupported competing state.
413	File exceeds upload size limit.
429	Verification/account requests exceed cooldown or attempt window.
503 / 500	Email unavailable, recovery pending, reset busy or infrastructure error; friendly production envelope.

The web API helper handles JSON and blob responses and passes HTTP status to callers: [api.js:21](#). AuthContext clears a stored session on authentication rejection, but retains it for temporary load failure and provides retry: [AuthContext.jsx:1](#). Mobile `_send` applies a 20-second timeout and clears a current session only after 401:

CareSync_mobile/lib/api_services.dart:32 (local working copy). Show unavailable/retry states instead of claiming the data is empty when a request failed.

Health at `/health` reports whether Mongoose is connected: [server.js:37](#). It is a database-connectivity signal, not a full file/email/workflow health probe. Required monitoring evidence should include relevant request and event/delivery logs, without publishing raw credentials or confidential patient data.

11. Deployment, the mobile build and backup requirements

Deployed components, recorded 6 October 2026

Component	Recorded configuration / source
Web	caresync-inky.vercel.app , Vercel Hobby. vercel.json:3 selects Vite and dist.
Backend	caresync-api-uhvz.onrender.com , Render Free, Singapore, Node 24. render.yaml:7 defines backend build/start/health.
Shared data/files	Atlas caresync_app, with GridFS bucket medicalFiles. Both clients call /api; neither holds MongoDB credentials.
Email	Gmail API over HTTPS; secrets in Render environment settings. env.js:1 validates configuration.
Android	APK built with --dart-define=API_BASE_URL=https://caresync-api-uhvz.onrender.com/api; artifact build/releases/CareSync-1.0.0.apk. Physical phone test pending.

Sources: [deployment receipt](#), docs/deployment/free-hosting-guide.md. This guide records that earlier deployment; it did not poll provider status again.

Backend-only settings: MONGO_URI, JWT_SECRET, GMAIL_CLIENT_ID, GMAIL_CLIENT_SECRET, GMAIL_REFRESH_TOKEN, sender address and optional SMTP password. UPLOAD_STORAGE, EMAIL_TRANSPORT, timezone, trusted proxy and CORS origins control runtime behavior. render.yaml uses unset private inputs rather than literal credentials.

Public build settings: only the API base URL is needed in Vercel/Flutter. Never put backend secrets in VITE_* values or Dart defines compiled into the client. Exact browser origins are allowed by CORS; requests without an Origin header still require normal API authentication. CORS is not a replacement for authorization.

The APK has a valid v2 signature using the existing development signing configuration and package com.example.flutter_care_sync; it is a demo sideload artifact, not a completed Play Store release. Hash: c44b3b8147fd01454be2c1dd82a3ac94d52c6f51d383f527d99199ffe8695d5d. Install by transferring the APK to Android; USB debugging is unnecessary for sideloading. Opening a mobile browser tests the website, not the Flutter APK.

Operational follow-up: link Vercel's GitHub project connection for automatic deployments; verify Render integration; test cold-start/retry on the phone; rotate exposed credentials in provider/Render settings. Do not assert automatic push-to-deploy already works. Free backend wake-up can exceed the current mobile timeout; check health before the defense.

Backup and recovery are separate pending requirements

The specification's documentation chapter 10.3 requires backup of **source code, database, uploaded files and documentation**; chapter 10.4 requires recovery steps. Its final submission also requires backup/recovery evidence. No completed full database backup/restore is recorded. Assignment compensation and Clear Demo Data do not satisfy this requirement.

Planned workflow only; no setup or execution was performed for this guide:

1. Select a quiet demo window and record the exact code revision, documentation version, database and timestamp. Stop clinic writers while making a consistent application snapshot; a basic standalone dump is not being claimed as an online point-in-time snapshot.
2. Dump the **whole** application database with MongoDB Database Tools. Include all collections, especially medicalFiles.files and medicalFiles.chunks; document metadata alone does not contain file bytes. Retain indexes/collection metadata from the dump.
3. Preserve matching source and docs. GitHub plus a dated repository archive/bundle protects the web/backend revision; separately save/commit the mobile working copy. A web repository backup does not include the sibling mobile repository. Keep required signing/recovery secrets separately protected, outside public source/docs.
4. Generate a checksum, encrypt the backup archive, and upload it to a private Google Drive folder. Drive stores the backup; it does not create a database export. Keep a local copy and the decryption password separately, with a documented owner and retention policy.

- 5. Restore into a new isolated test database, leaving production untouched. Verify accounts/login with matching configuration, counts, relationships, versions, comments, audit/inbox records and current/history file hashes/downloads. Prevent restored notification/queue handlers from sending messages or mutating production during verification.
- 6. Save sanitized dump/restore logs, screenshots, file verification results and recovery timings. State actual results and observed recovery time; do not invent an RPO/RTO achievement.

Exact commands and network settings should be finalized when the user resumes backup work. The existing maintenance scripts concern migration/reconciliation, not a configured recurring Drive backup job. No backup scheduler or Drive upload integration is implemented.

Clear Demo Data: actual deletion behavior

Source: [demoReset.service.js:36](#). Full path: backend/src/services/demoReset.service.js.

```
36     deleted.User = (await User.deleteMany({ role: { $nin: ['Doctor', 'Staff'] }, _id: { $ne: adminId }
37   })).deletedCount;
37     if (env.UPLOAD_STORAGE === 'gridfs' || mongoose.connection.db) await clearStoredFiles();
38     // Fixed application-owned directory, never a path supplied by the caller.
39     await fs.rm(UPLOAD_ROOT, { recursive: true, force: true });
40     await fs.mkdir(UPLOAD_ROOT, { recursive: true });
```

It retains Doctor accounts, Staff accounts and the requesting Admin; it deletes Patient accounts and other Admin accounts, clinic records, notifications/audit/verification/recovery data and uploaded files. The API requires Admin plus explicit confirmation: [system.routes.js:50](#). Its in-process guard checks active requests and pending handlers. It is destructive cleanup, not a backup. Do not reset before collecting required evidence or preserving data you intend to keep.

12. Testing: what exists, what passed and what still needs evidence

The deployment receipt records all **17 backend check files passed** after dependency fixes; a production dependency audit reported zero known vulnerabilities at that time. These are earlier isolated checks, not a rerun during document preparation. Prior browser evidence includes synthetic role screens and limited deployed landing/login checks; synthetic fixtures do not prove the live end-to-end workflow.

Case IDs below are a **submission checklist**, not fabricated executed results. Connect each to the assertion/output in the named check file, actual dated result and screenshot/log. If a chosen scenario is not asserted there, execute it before marking Pass. A file can contain many cases; 17 files is not the same measurement as the specification's per-category minimum.

Functional cases — required minimum 8	Relevant existing checks
F01 valid verified registration; F02 valid login	otp.check.js, accounts-noshow.check.js, api-access.check.js
F03 booking; F04 staff approval; F05 decline with reason	clinic-workflows.check.js, concurrency.check.js
F06 consultation note revision; F07 attachment/history	versions.check.js, file-storage.check.js
F08 appointment feedback comment	comments.check.js

Integration cases — required minimum 5	Relevant existing checks
I01 booking updates appointment/slot/history	concurrency.check.js, clinic-workflows.check.js
I02 review event persists inbox; I03 audit event persists actor/target	notifications.check.js, audit-coverage.check.js
I04 freed slot assigns eligible walk-in	walkin-status.check.js, concurrency.check.js
I05 Gmail transport accepts encoded message / records failure	gmail-api.check.js, email-notifications.check.js (mocked provider; live inbox proof separate)

Error-handling cases — required minimum 5	Relevant existing checks
E01 competing slot claim; E02 stale notes/file update	concurrency.check.js, versions.check.js
E03 expired/wrong/reused OTP; E04 unavailable email provider	otp.check.js, gmail-api.check.js
E05 invalid request/file/query	api-access.check.js, file-storage.check.js, deployment.check.js; also recorded live malformed JSON rejection

Security cases — required minimum 3	Relevant existing checks
S01 unauthenticated private endpoint denied	api-access.check.js, recorded live HTTP 401
S02 wrong role denied; S03 unrelated appointment/file owner denied	api-access.check.js, comments.check.js, versions.check.js

Source links: [api-access.check.js:1](#); [otp.check.js:1](#); [versions.check.js:1](#); [file-storage.check.js:1](#); [notifications.check.js:1](#); [audit-coverage.check.js:1](#); [gmail-api.check.js:1](#); [clinic-workflows.check.js:1](#).

The other check files cover accounts/no-show, corrections, concurrency, demo reset, deployment, email delivery outcomes, scheduled queue and walk-in statuses. All live evidence must be kept separate from stubbed/isolated results. Existing folders under docs/final_submission/testing-evidence/ contain dated outputs and screenshots; their presence does not mean every final assessment item is done.

The required one full end-to-end scenario remains pending

E2E01: on the real Android APK, patient receives verification code, registers/logs in and books; Staff reviews the same request on deployed web; patient sees confirmation/inbox; clinic checks in and starts the visit; assigned Doctor saves notes, uploads/replaces a demo file and completes; patient reads history/downloads; Admin sees matching audit; report/queue reflect the result. Check persistence after backend restart and compare file hashes. Use one appointment ID to correlate sanitized evidence across components.

Capture expected/actual behavior and pass/fail for each step. A cold-start retry or provider failure must be recorded, not omitted. No physical-phone execution, complete hosted clinical E2E or full database restore has yet been established in the available evidence.

13. Functional, quality and screen traceability

Assessable functional requirements

These IDs are this guide's cross-reference, not a silent renumbering of the existing paper's 17 FRs.

ID	Expected behavior / implementation
G-FR01	Verified Patient registration and password reset. <code>user.controller.js</code> , <code>user.service.js</code> , <code>otp.service.js</code> ; sections 4–5.
G-FR02	Login and role/status management. <code>auth.js</code> , <code>user.routes.js</code> , <code>App.jsx</code> ; section 4.
G-FR03	Book against real schedule capacity or request an unassigned booking. <code>appointment.service.js:create</code> , <code>slot.service.js:reserveSlot</code> ; section 6.
G-FR04	Staff/Admin approval, decline, correction and patient resubmission. <code>appointment.routes.js</code> , <code>reviseBooking</code> ; section 6.
G-FR05	Check-in/consultation/completion, cancellation and no-show history. <code>checkIn</code> , <code>updateConsultation</code> , <code>cancel</code> , <code>noShow</code> ; sections 6–7.
G-FR06	Doctor availability, recurring slot plans and walk-in assignment. <code>slot.service.js</code> , <code>walkIn.service.js</code> ; section 7.
G-FR07	Medical uploads and authorized downloads. <code>upload.js</code> , <code>downloadDocument</code> ; section 8.
G-FR08	Content versions and archived attachment history. <code>uploadDocuments</code> , <code>attachFile</code> , <code>getVersions</code> ; section 8.
G-FR09	Appointment discussion with author/time. <code>AppointmentComment</code> , <code>addComment</code> ; section 9.
G-FR10	Inbox/read state and selected patient email delivery. <code>notification.routes.js</code> , <code>notification.service.js</code> , <code>email.service.js</code> ; section 9.
G-FR11	Admin audit trail. <code>auditLog.handler.js</code> , <code>auditLog.routes.js</code> ; section 9.
G-FR12	Role dashboards/reports and public anonymized queue. <code>App.jsx</code> , <code>report.service.js</code> , <code>ClinicDisplayScreen.jsx</code> ; sections 7, 10.

Quality requirements: minimum eight, ten mapped here

ID / quality	Implementation and verification boundary
G-NFR01 Authentication/authorization	JWT, current account/role, ownership and route roles. Isolated access checks and live no-token rejection recorded; no full penetration test.
G-NFR02 Credential/input protection	bcrypt/HMAC, environment secrets, private upload route, request/file validation. Extension filter is not malware scanning.
G-NFR03 Privacy	Own/assigned appointment scope, minimal public queue, private inbox, limited doctor list. Staff/Admin broad access and backup handling need policy.
G-NFR04 Consistency	Unique indexes, conditional writes, revision checks and compensation journal. Not a global multi-collection ACID guarantee.
G-NFR05 Fault handling	Friendly JSON failures, preserved sessions on outages, inbox retained if email fails. Process-local events may still be lost.
G-NFR06 Usability/accessibility	Labeled forms, Show/Hide password, shared modal/error states and responsive styles. Prior UI checks exist; formal accessibility/device audit pending.
G-NFR07 Compatibility	React browser client and Flutter Android client use one API. APK builds; actual phone/version/network test pending.
G-NFR08 Maintainability	Layered modules, shared helpers, check scripts and configuration manifests. Code links/hashes make documentation reviewable.
G-NFR09 Performance/availability	Indexed lookups, aggregate reports, bounded upload/OTP state and health route. No measured load SLA; free hosting sleeps.
G-NFR10 Backup/recoverability	Whole DB+GridFS, source/mobile/docs archive and isolated restore plan. Implementation/execution/evidence still pending; not marked met.

UI sources: [LoadError.jsx:1](#); [DialogContext.jsx:1](#); [responsive.css:1](#). These support implementation descriptions, not automatic WCAG certification.

The twelve required screens and their equivalents

Specification screen	Actual web counterpart / code
Login	LoginPage.jsx:1 ; sign-in modal in App.jsx.
Dashboard	PatientDashboard.jsx:1 , DoctorDashboard, StaffDashboard, AdminDashboard.
Project List	Appointment lists in role dashboards; healthcare work-item equivalent.
Project Details	AppointmentPatientDetails.jsx:1 and StatusTimelineModal.jsx:1 .
Submission Form	ConsultationModal.jsx:1 upload/categorization/notes.
Version History	VersionHistory.jsx:1 .
Review / Approval	Staff dashboard actions and BookingCorrections.jsx:1 .
Comment / Feedback	AppointmentComments.jsx:1 .
Notification	NotificationInbox.jsx:1 .
Audit Log	AuditLogViewer.jsx:1 .
Reports	ReportsPage.jsx:1 .
User / Role Management	AdminDashboard.jsx:1 and AddUserModal.jsx:1 .

Mobile is the patient portal rather than four clinic-role dashboards. Relevant local screens:

CareSync_mobile/lib/login_screen.dart:1 (local working copy), **CareSync_mobile/lib/register_screen.dart:1** (local working copy), **CareSync_mobile/lib/appointments_screen.dart:1** (local working copy), **CareSync_mobile/lib/appointment_detail_screen.dart:1** (local working copy), **CareSync_mobile/lib/notifications_screen.dart:1** (local working copy). Confirm screen requirements can be satisfied by these pages/dialogs rather than assuming twelve independent URLs are mandatory.

14. Required paper chapters, diagrams and submission evidence

Connect the 15-chapter paper to this code guide

Required chapter / subsections	Where implementation support comes from / remaining writing
Title page	Insert actual group members, course, instructor, institution and date. No names invented.
1: 1.1–1.5 background, problem, objectives, scope, users	Healthcare scope and component mapping: sections 1–2. Confirm actual clinic observations.
2: 2.1–2.3 current process, issues, stakeholders	Current-state draft and stakeholder register; validate interviews/observations rather than treating assumptions as research findings.
3: 3.1–3.4 FR, NFR, interfaces, traceability	Sections 3 and 13; reconcile IDs with the paper and final test register.
4: 4.1–4.4 business, data, application, technology architecture	Sections 3, 6–7 and 11. Explain clinical flow, schemas, layers and deployed platforms.
5: 5.1–5.5 target process, architecture diagram, style/pattern comparison, decisions	Section 3 plus current technical choices: modular layers, common API, GridFS, Gmail and compensation. Add formal decision records if required.
6: 6.1–6.6 use case, context, DFD, sequence, ERD and UI	Diagram register below and section 13. Update drawings to match current implementation.
7: 7.1–7.6 scenario, integration, mapping, payload, validation, logs	Sections 3, 6, 8–10; use one actual demo transaction for request/database/event evidence.

Required chapter / subsections	Where implementation support comes from / remaining writing
8: 8.1–8.6 access matrix, least privilege, privacy, audit, governance, risks	Sections 4–5 and 9–11. Assign real owners/retention and maintain at least eight assessed risks; existing paper has 11 preliminary risks.
9: 9.1–9.6 plan, functional/integration/error/E2E cases and results	Section 12 and dated check outputs. Record case-level expected/actual results and complete live E2E.
10: 10.1–10.4 deployment, evidence, backup and recovery	Deployment receipt plus section 11. Full dump and isolated restore remain unexecuted.
11: 11.1–11.3 demo, sample accounts/data	Prepare authorized role accounts and nonconfidential data; rehearse the full flow. Keep passwords outside public documents.
12: 12.1–12.2 limitations/enhancements	Sections 3–13 identify actual limits; propose durable outbox, stronger production policy, monitoring and backup automation separately.
13: Group contributions	Record actual work, owners and evidence.
14: References	Specification, pinned source, provider/tool documentation and dated evidence; use the instructor's citation style.
15: Appendices	Case results, screenshots, payloads/logs, schema/config examples, backup/restore proof and this code guide.

Required diagrams: source files and accuracy checks

Existing files are under `docs/final_submission/diagrams/`. This guide maps them; it does not assert they were redrawn during this task.

Diagram	Editable/output file and code relationship
Current-state process	01-current-state.mmd/.svg. Clinic baseline requires confirmation; this is not inferred from the application's code.
Target-state process	02-target-state.mmd/.svg. Booking/review/check-in/consultation flow from <code>appointment.service.js</code> .
Context	03-context.mmd/.svg. Patient/clinic clients, API, MongoDB and Gmail external boundary.
UML use case	04-use-case.svg. Four roles and permissions from route/auth controls.
Data flow	05-data-flow.mmd/.svg. HTTP data → service → MongoDB/history → notifications/audit; include GridFS byte storage.
Sequence	06-sequence.mmd/.svg. Align booking/approval sequence with compensation and asynchronous reactions; do not imply atomic delivery.
Application architecture	07-application-architecture.mmd/.svg. Both clients, layered backend, event handlers, MongoDB and Gmail.
Deployment	08-deployment.mmd/.svg. Existing drawing reflects local development; update to Vercel, Render, Atlas/GridFS and Gmail HTTPS API.
Additional ERD/database	09-erd.mmd/.svg. Models/references plus version arrays, notification delivery, OTP and recovery/GridFS where absent.

The architecture drawings and paper must agree with this guide: uploaded bytes are in GridFS for production; current queue refresh is polling; Gmail uses HTTPS; internal events are not a distributed broker. A GitHub source link alone does not establish a live workflow or diagram correctness.

Final submission readiness

Item	State / action needed
Paper printed + PDF	Draft exists; complete human details, required subsection corrections and updated implementation/evidence claims.

Item	State / action needed
Working prototype/link	Web/API deployment recorded; install/check Android APK and complete private clinical workflow.
Source code	Web/backend pinned; mobile working changes need separate preserved/submitted source.
DB file or migration material	Existing models and migration scripts explain structure; create sanitized reproducible DB/export material and backup according to submission requirements.
Feature screenshots	Existing synthetic UI captures available; collect real completed feature evidence with demo data.
Integration proof	Isolated tests/live HTTP available; capture one correlated client/API/database/notification/audit transaction.
Test evidence	Earlier suites passed; finalize case register and full E2E result.
Backup/recovery evidence	Pending whole-database dump, code/mobile/docs archive, isolated restore and verification.
Slides	Prepare concise demo/architecture/decision/limitations slides.
Contributions	Group must supply accurate names, roles and actual contributions.

15. How to use this guide in the technical defense

Explain each technical feature through **purpose** → **request** → **server rule** → **stored change** → **observable result** → **limitation**. Open the pinned source link or show the short excerpt, then demonstrate the matching behavior with demo data.

Likely defense question	Answer grounded in this implementation
"Where is the integration?"	React and Flutter use the same REST API. Clinical writes update shared MongoDB records; events create inbox/audit reactions; the backend calls Gmail over HTTPS. Show section 3's controller, section 9's transport and matching evidence.
"Why isn't this just CRUD?"	Booking claims scarce schedule capacity, approval/corrections drive state transitions, released slots can assign walk-ins, records retain content/history and actions generate notifications/audit. Show sections 6–9.
"Can someone read another patient's file?"	Central JWT plus appointment ownership and role middleware run before protected download. Raw storage paths return 404. Demonstrate the forbidden request with two demo patients.
"What happens if two users book together?"	Conditional Available/unowned slot update admits one claim; the other gets 409. Recovery journal compensates failed assignment writes. It is not a global database transaction.
"What happens when Gmail is down?"	Verification returns unavailable; clinical inbox survives an email failure, with failed delivery recorded. Events/email are not guaranteed replayed after a crash.
"How are versions real?"	Notes and file snapshots store prior content/storage references, authors and times. Archived attachments are queryable through protected version downloads. Show a replacement and the old/new bytes.
"How does mobile work after deployment?"	Compile the hosted API URL into Flutter; phone communicates with Render, sharing Atlas with web. No USB debugging is needed to install the APK. Device E2E still must be recorded.
"Can we restore the database?"	A plan exists, but full dump/isolated restore is not yet demonstrated. Include GridFS bytes and matching code/mobile/docs, then report verified recovery results after execution.

Before marking the project fully complete: confirm theme/screen interpretation; update the main paper/diagrams; install/test the APK; record real Gmail and clinical E2E; perform the separately authorized backup/restore workflow; fill case results, group details and slides. This guide provides the implementation explanation and code traceability, not invented completion evidence.

Companion artifacts: technical-specification-code-guide.md is editable; .html is readable offline; .pdf is printable; technical-specification-source-index.json records reviewed file hashes, exact citation anchors and excerpt lines. Runtime application code was not changed to prepare these documents.